SQL Temel Komutları & CRUD Prensibi

CRUD Prensibi

CRUD, veri tabanında yapılabilecek **temel dört işlemi** temsil eder:

Harf	İşlem	SQL Komutu
C	Create (Oluştur)	INSERT
R	Read (Oku)	SELECT
U	Update (Güncelle)	UPDATE
D	Delete (Sil)	DELETE

🔹 CRUD Örnekleri

Create (INSERT): Veri ekler.

```
INSERT INTO musteriler (id, ad, yas, email)
VALUES (3, 'Ayşe', 22, 'ayse@example.com');
```

Read (SELECT): Verileri okur.

```
SELECT * FROM musteriler;           -- tüm sütunları getir
SELECT ad, email FROM musteriler;   -- sadece ad ve email süt
unlarını getir
```

Update (UPDATE): Veri günceller.

```
UPDATE musteriler
SET yas = 26
WHERE id = 1;
```

Delete (DELETE): Veri siler.

```
DELETE FROM musteriler
WHERE id = 2;
```

📌 **Not:**

- **Update** ve **Delete** işlemlerinde yanlışlıkla bir veriyi silmemek veya güncellememek için `"id"` üzerinden işlem yapılır!

SQL Temel Komutları

SQL komutları genellikle **4 ana gruba** ayrılır:

1️⃣ DDL (Data Definition Language) – Veri Tanımlama

Veri tabanı ve tabloların yapısını tanımlamak için kullanılır. (**CREATE, ALTER, DROP.)**

- **CREATE** → Yeni veritabanı veya tablo oluşturur.

```
CREATE DATABASE okul;
CREATE TABLE ogrenciler (
    id INT PRIMARY KEY,
    ad VARCHAR(50),
    yas INT
);
```

- **ALTER** → Tablo yapısını değiştirir (sütun ekleme, değiştirme).

```
ALTER TABLE ogrenciler ADD email VARCHAR(100);
```

- **DROP** → Veritabanı veya tabloyu siler.

```
DROP TABLE ogrenciler;
```

SQL'de CREATE kullanımından önce oluşturacağımız tablonun veri tabanında olup olmadığını nasıl anlarız ?

🏗️ CREATE TABLE IF NOT EXISTS

Tanım

- `CREATE TABLE` → SQL'de yeni bir tablo oluşturmak için kullanılır.
- `IF NOT EXISTS` → Tablonun zaten var olup olmadığını kontrol eder.
- Eğer tablo **varsa**, SQL **hata vermez** ve tabloyu tekrar oluşturmaz.
- Eğer tablo **yoksa**, tablo oluşturulur.

Temel Söz Dizimi:

```
CREATE TABLE IF NOT EXISTS tablo_adi (
    sutun1 veri_tipi [kısıtlamalar],
    sutun2 veri_tipi [kısıtlamalar],
    ...
);
```

Kullanım Örneği

Örnek Tablo: `musteriler`

```
CREATE TABLE IF NOT EXISTS musteriler (
    id INT PRIMARY KEY,
    ad VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE,
    yas INT,
    kayit_tarihi DATE
);
```

Açıklamalar:

- `IF NOT EXISTS` → Eğer `musteriler` tablosu varsa SQL hata vermez
  - Eğer tablo zaten varsa **mevcut tabloya dokunulmaz**.
  - Eğer tablo yoksa oluşturulur.
- `id INT PRIMARY KEY` → Birincil anahtar ve tam sayı.
- `ad VARCHAR(50) NOT NULL` → Ad sütunu boş bırakılamaz.
- `email VARCHAR(100) UNIQUE` → Her email benzersiz olmalı.
- `yas INT` → Yaş bilgisi.
- `kayit_tarihi DATE` → Kayıt tarihi.

Neden Kullanılır?

1. **Hata önleme:**

- Aynı tabloyu tekrar tekrar oluşturmaya çalışırken oluşacak hatayı engeller.

2. **Kodun güvenli çalışması:**
   - Özellikle script veya migration dosyalarında tablo kontrolü sağlar.
3. **Taşınabilirlik:**
   - Farklı veri tabanlarında veya tekrar eden çalıştırmalarda güvenli bir yöntemdir.

2️⃣ DML (Data Manipulation Language) – Veri İşleme Komutları

Tablolar içindeki **verilerle çalışmak** için kullanılır. **(INSERT, SELECT, UPDATE, DELETE)**

- CRUD Prensibi veri işleme komutları için geçerlidir.
- Örnekler CRUD prensibinde anlatılmıştır.👆

3️⃣ DCL (Data Control Language) – Yetkilendirme Komutları

Kullanıcıların veritabanı üzerindeki yetkilerini kontrol eder.

- **GRANT** → Kullanıcıya yetki verir.

```
GRANT SELECT, INSERT ON okul.* TO 'ahmet'@'localhost';
```

- **REVOKE** → Kullanıcıdan yetki alır.

```
REVOKE INSERT ON okul.* FROM 'ahmet'@'localhost';
```

4️⃣ TCL (Transaction Control Language) – İşlem Kontrol Komutları

Birden fazla sorgunun tek işlem olarak çalışmasını sağlar.

- **BEGIN / START TRANSACTION** → İşlem başlatır.
- **COMMIT** → İşlemi onaylar (kalıcı yapar).
- **ROLLBACK** → İşlemi geri alır.

```
START TRANSACTION;
UPDATE ogrenciler SET yas = 27 WHERE id = 1;
ROLLBACK;   -- değişiklik geri alınır
COMMIT;     -- değişiklik onaylanır
```

➡️ Veri Çekme (SELECT) Komutuyla Çalışma

Koşullu sorgular:

```
--Tüm müşterileri listele:
SELECT * FROM musteriler;

--Belirli sütunları seç:
SELECT ad, email FROM musteriler;

-- Yaşı 25'ten büyük müşterileri getir:
SELECT * FROM musteriler WHERE yas > 25;

--İsim desenine göre arama → B harfi ile başlayan müşterileri
getir:
```

```
SELECT * FROM musteriler WHERE ad LIKE 'B%';

--Belirli değerler → 20,25,30 yaş bilgisi içeren müşterileri
getir:
SELECT * FROM musteriler WHERE yas IN (20, 25, 30);

--Aralık sorgusu → Yaşı 18 ve 25 arasındaki müşterileri geti
r:
SELECT * FROM musteriler WHERE yas BETWEEN 18 AND 25;
```

Sıralama ve sınırlandırma:

```
--Yaşa göre sırala → Yaşı azalan şekilde sırala(büyükten küçü
ğe):
SELECT * FROM musteriler ORDER BY yas DESC;

--Yaşa göre sırala → Yaşı artan şekilde sırala(küçükten büyüğ
e):
SELECT * FROM musteriler ORDER BY yas DESC;

--İlk 5 kayıtı, müşteriyi getir:
SELECT * FROM musteriler LIMIT 5;
```

Gruplama ve özetleme:

```
--Toplam sipariş adedi 5’ten fazla olan ürünleri listelemek:

SELECT urun, SUM(adet) AS ToplamAdet
FROM siparisler
GROUP BY urun       -- Ürünleri gruplandır
HAVING SUM(adet) > 5;    --Belirli koşulu ekle.
```

📌 **Not:**

Matematiksel işlemler veya toplu değerler üzerinde koşullu filtreleme yapmak için **HAVING** kullanılır.

- **GROUP BY** ile gruplama işlemi yaptıktan sonra **HAVING** ile grupları filtreleyebilirsiniz.

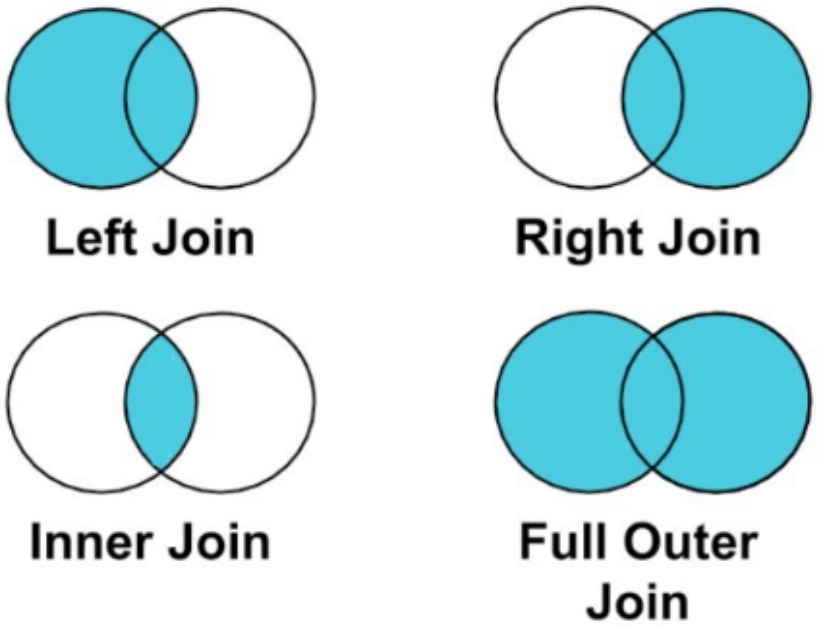
🔹 **Önemli: HAVING**, WHERE ile karıştırılmamalıdır; `WHERE` **satır bazlı** filtreleme yaparken, `HAVING` **grup bazlı** filtreleme yapar.

Tabloları birleştirme (JOIN):

Tanım:

- JOIN, **iki veya daha fazla tabloyu ortak bir sütun üzerinden birleştirmek** için kullanılır.
- Amaç: Farklı tablolardaki ilgili verileri **tek bir sonuç kümesinde** görmek

JOIN Türleri



JOIN Türü	Açıklama
LEFT JOIN (LEFT OUTER JOIN)	Sol tablodaki tüm kayıtları ve sağ tablodaki eşleşenleri getirir. Eşleşmeyen sağ tablo değerleri NULL olur.
RIGHT JOIN (RIGHT OUTER JOIN)	Sağ tablodaki tüm kayıtları ve sol tablodaki eşleşenleri getirir. Eşleşmeyen sol tablo değerleri NULL olur.
INNER JOIN	İki tabloda da eşleşen kayıtları getirir.
FULL JOIN (FULL OUTER JOIN)	Her iki tablodaki tüm kayıtları getirir, eşleşmeyen kısımlar NULL olur.

Örnek Senaryo

Tablolar:

`musteriler`

id	ad	email
1	Büşra	busra@example.com
2	Ahmet	ahmet@example.com
3	Ayşe	ayse@example.com

`siparisler`

id	musteri_id	urun	adet
1	1	Laptop	2
2	1	Mouse	1
3	2	Klavye	1
4	4	Monitör	1

INNER JOIN Örneği

- Sadece **eşleşen kayıtları** getirir:

```
SELECT m.ad, s.urun, s.adet
FROM musteriler m
INNER JOIN siparisler s ON m.id = s.musteri_id;
```

Sonuç:

ad	urun	adet
Büşra	Laptop	2
Büşra	Mouse	1
Ahmet	Klavye	1

> Ayşe ve siparişi olmayan veya siparişi olan ama müşteri kaydı olmayan `(siparis id=4)` kayıtlar **gelmez**.

LEFT JOIN Örneği

- Sol tablodaki tüm müşterileri, siparişleri varsa getirir:

```
SELECT m.ad, s.urun, s.adet
FROM musteriler m
LEFT JOIN siparisler s ON m.id = s.musteri_id;
```

Sonuç:

ad	urun	adet
Büşra	Laptop	2
Büşra	Mouse	1
Ahmet	Klavye	1
Ayşe	NULL	NULL

> Ayşe'nin siparişi olmadığı için **NULL** değerler gelir.

📌 **Not:**
SQL'de tablo birleştirme (JOIN) yaparken **birden fazla tablo aynı sütun adlarına sahip olabilir**. Bu durumda SQL'e hangi tablodan veri çektiğini belirtmek gerekir.

Kısaltmalar (Alias) Kullanımı

Örneğimizde:

- `m` → `musteriler` tablosu için **kısa isim (alias)**
- `s` → `siparisler` tablosu için **kısa isim (alias)**

Neden kullanılır:

1. **Kısaltma** sayesinde kod daha okunaklı olur. Uzun tablo isimleri tekrar tekrar yazılmaz.
2. **Çakışmayı önler**: Örneğin `id` her iki tabloda da var, `m.id` veya `s.id` diyerek hangi tablodan geldiğini netleştiririz.

Seçilen Sütunlar

- `m.ad` → `musteriler` tablosundaki `ad` sütunu
- `s.urun` → `siparisler` tablosundaki `urun` sütunu
- `s.adet` → `siparisler` tablosundaki `adet` sütunu

> Yani `s.urun` ve `s.adet` s sütun alias'ı üzerinden `siparisler` tablosundan gelen verileri ifade eder. `m.ad` ise, m sütun alias'ı üzerinden `musteriler` tablosundan gelen verileri ifade eder.

❗ Özet

- **Alias (m, s)**: Tabloyu kısa isimle temsil eder.
- **m.ad, s.urun**: Hangi tablodan veri alındığını belirtir.
- JOIN'lerde tablo isimleri belirtilmezse, **sütun çakışmalarında hata oluşur.**

📌 Önemli Not:

Veri analisti olarak SQL bilginiz, bir veri tabanı uzmanı kadar kapsamlı olmak zorunda değildir.

Önemli olan, **veri işleme ve sorgulama odaklı CRUD komutlarını (INSERT, SELECT, UPDATE, DELETE)** etkin şekilde kullanabilmenizdir.

Özellikle veri analizi sürecinde, şu beceriler önceliklidir:

- Veriyi **çekmek ve filtrelemek** (WHERE, LIKE, IN, BETWEEN)
- Veriyi **sıralamak ve sınırlamak** (ORDER BY, LIMIT)
- Veriyi **gruplamak ve özetlemek** (GROUP BY, HAVING, COUNT, SUM, AVG)
- Farklı tabloları **birleştirerek anlamlı bilgi elde etmek** (JOIN)

Bu yetkinlikler, veri analistinin günlük raporlama, veri keşfi ve analiz görevlerini etkili bir şekilde gerçekleştirmesini sağlar.